



# High-Performance Computer Architectures Practical Course

OpenCL

**Prof. Dr. Ivan Kisel**

Robin Lakos

Akhil Mithran

Oddharak Tyagi

June 17, 2025

OpenCL : “**Open** Computing Language”

- Standard for general purpose parallel programming for CPUs, GPUs and others
- Allows portability between CPU/GPU
- Provides low-level, high-performance portable abstraction

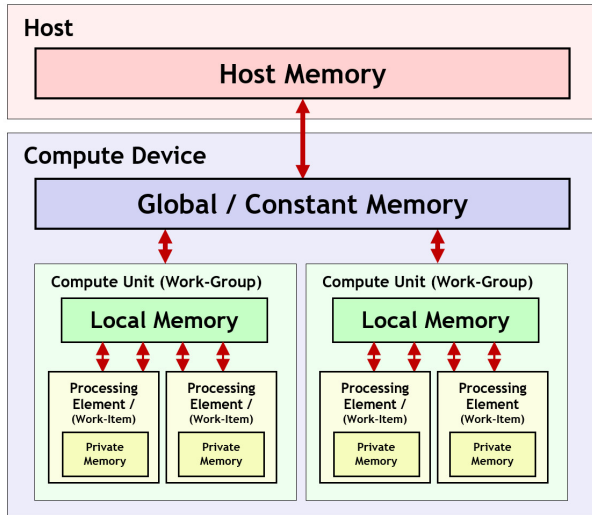


Figure: Memory Model in OpenCL

# General Structure of Host Program

## Structure

- get a platform / device - the information about the whole / select the device for computations
- set a context within which the program will work
- create a command-queue
- create memory buffer(s)
- write the buffer (fill the buffer with the input data)
- create a program - an OpenCL object, the input for it - a \*.cl file with the main kernel function
- create a kernel (with kernel program as argument)
- call the kernel (each work-item runs the kernel)
- read the buffer
- clean the memory

## Example

```
__kernel void vectorAdd(__global int *A, __global int *B, __global int *C) {  
  
    // Get the index of the current element  
    int i = get_global_id(0);  
  
    // Do the operation  
    C[i] = A[i] + B[i];  
}
```

- The workitems ("threads") are organized in a NDRange (up to 3 dimensions).
- Each thread is mapped to the piece of data it will have to manipulate.
- Therefore, each thread is uniquely identified
- Use workitem built-in functions to know which workitem it is and where it stands in the NDRange.

**NDRange dimensions:** Know how many dimensions the NDRange is made of

```
uint get_work_dim();
```

**The global unique identifiers:** Each thread is uniquely identified with indices corresponding to the specific dimensions

```
size_t get_global_id (uint Dimension)
```

**The global size:** Query how big is the NDRange for a specific dimension

```
size_t get_global_size (uint Dimension)
```

Simple element-wise addition of two arrays:

```
void kernel addVector( global const int* a, global const int* b, global int* c){  
    int gid = get_global_id(0);  
    c[gid] = a[gid] + b[gid];  
}
```

Simple element-wise addition of two arrays with batching:

```
void kernel addVector(global const int* a,
                      global const int* b,
                      global int* c,
                      const int N,
                      const int batchSize)
{
    int gid = get_global_id(0) * batchSize;
    for (int i = 0; i < batchSize; ++i) {
        int idx = gid + i;
        if (idx < N) {
            c[idx] = a[idx] + b[idx];
        }
    }
}
```

Illustration of arranging work items in multi-dimensions:

```
void kernel multiDim2OneDim( global const int* a,
                             global const int* b,
                             global const int* c,
                             global int* d,
                             int dimN1,
                             int dimN2,
                             int dimN3)
{
    int dim1 = get_global_id(0);
    int dim2 = get_global_id(1);
    int dim3 = get_global_id(2);

    int index = dim1 + (dimN1*dim2) + (dimN1*dimN2*dim3);

    d[index] = a[dim1] + b[dim2] + c[dim3];
}
```